Dual-head TensorFlow — struttura dei layer

Ogni box = layer Keras reale · flusso dall'alto verso il basso

1 · INGRESSI

Embedding categorici
13 Input int32
Embedding(n, dim)→Flatten
sku:12 · pdv:8 · …

Bande / regimi
mq_espo · mq_stock
weather_regime
Embedding band

Numeriche
~57 float + z-score
BatchNorm num_bn
RR, meteo, sconto…

sku_prior
run_rate_14d × H
salta il tronco
→ Multiply in testa

Concatenate
embeddings ■ numeric_bn → vettore condiviso

2 · TRONCO RESIDUALE CONDIVISO

Residual block #1 · 256
Dense(256)→BN→ReLU→Drop(0.15)
Dense(256)→BN → Add(skip) → ReLU

Residual block #2 · 256
Dense(256)→BN→ReLU→Drop(0.15)
Dense(256)→BN → Add(skip) → ReLU

Residual block #3 · 128
Dense(128)→BN→ReLU→Drop(0.10)
Dense(128)→BN → Add(skip*) → ReLU

Zoom: un residual block
x ■■■
Dense → BN → ReLU → Drop
Dense → BN
x ■■■■■ Add → ReLU → out

* se dim cambia: Dense sulla shortcut (no bias)

prior (byp

3 · DUE TESTE

TESTA SKU (output operativo)
Dense(64) ReLU → Drop(0.10)
■■ sku_scale Softplus (init ≈ 1)
■■ sku_delta lineare (init = 0)

y_sku = ReLU(prior × scale + delta)
Loss Huber δ=5 · peso 1.0

TESTA CATEGORIA (solo train)
Lambda stop_gradient(x)
→ non aggiorna il tronco
Dense(64) ReLU → Drop(0.10)
y_cat = Softplus(Dense(1))

target: log1p(totale cat)
Loss Huber δ=1 · peso 0.15

Formula SKU: pezzi = ReLU(sku_prior × softplus(scala) + delta)

LightGBM — struttura dell'ensemble ad alberi

Non è una rete a layer: è una foresta di alberi di decisione addestrata in boosting

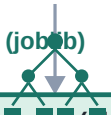
Ogni albero corregge l'errore dei precedenti. In deploy ci sono DUE booster + un mix.

1 · MATRICE FEATURE (stesso schema del TF, ma tabulari)

X = NUMERIC_COLS (~57) + sku_prior + CAT_COLS (categorie + bande mq/meteo)

categorie passate come categorical_feature (split su livelli, non one-hot)
target y = pezzi sull'orizzonte H (15 / 30 / 60 / 90 giorni)

2 · DUE MODELLI LGBM (joblib)



A · Tweedie (globale)

objective = tweedie (p=1.3)
n_estimators ≤ 2500 (early stop)
num_leaves = 95
lr = 0.04 · λ = 1.2
subsample / colsample = 0.85

pred_tw = max(predict(X), 0)
copre tutta la distribuzione
(molti zeri + coda lunga)

B · High-volume (specialista)

objective = regression (su log1p)
train solo se prior ≥ soglia
n_estimators ≤ 1500 · leaves=63
lr = 0.05

pred_hi = expm1(predict(X))
affina i volumi alti
(dove Tweedie sottostima)

Anatomia di UN albero LightGBM (leaf-wise)

radice → split su feature (es. run_rate_14d < 0.8 | sku_id ∈ {…} | temp_media ≥ 22)
→ foglie con valore numerico (contributo additivo alla previsione)

predizione finale di un booster = somma dei contributi di tutti gli alberi
F(x) = F■ + Σ■ η · tree■(x) η = learning_rate
early stopping: si ferma quando la MAE di validazione non migliora (100 / 80 round)

3 · BLEND DI DEPLOY (nativi h30/h60/h90 e ramo LGBM del 15g)

Mix volume-aware

prior = run_rate_14d × H
se prior basso → solo Tweedie
se prior alto → 0.55·Tweedie + 0.45·High e floor ≥ 0.35·naive
Sul 15g l'ensemble completo aggiunge anche Dual-head + routing calibrato per fascia di prior